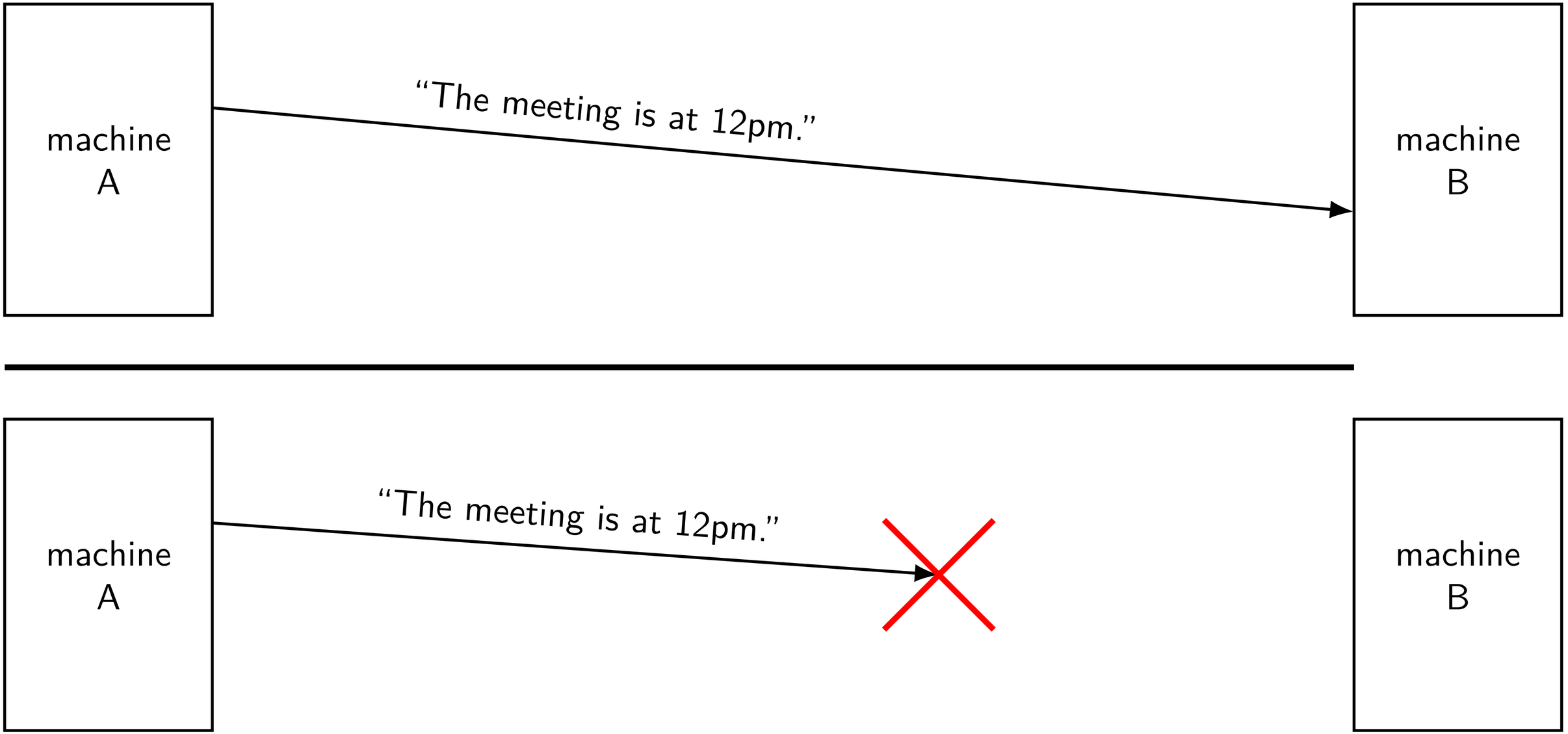
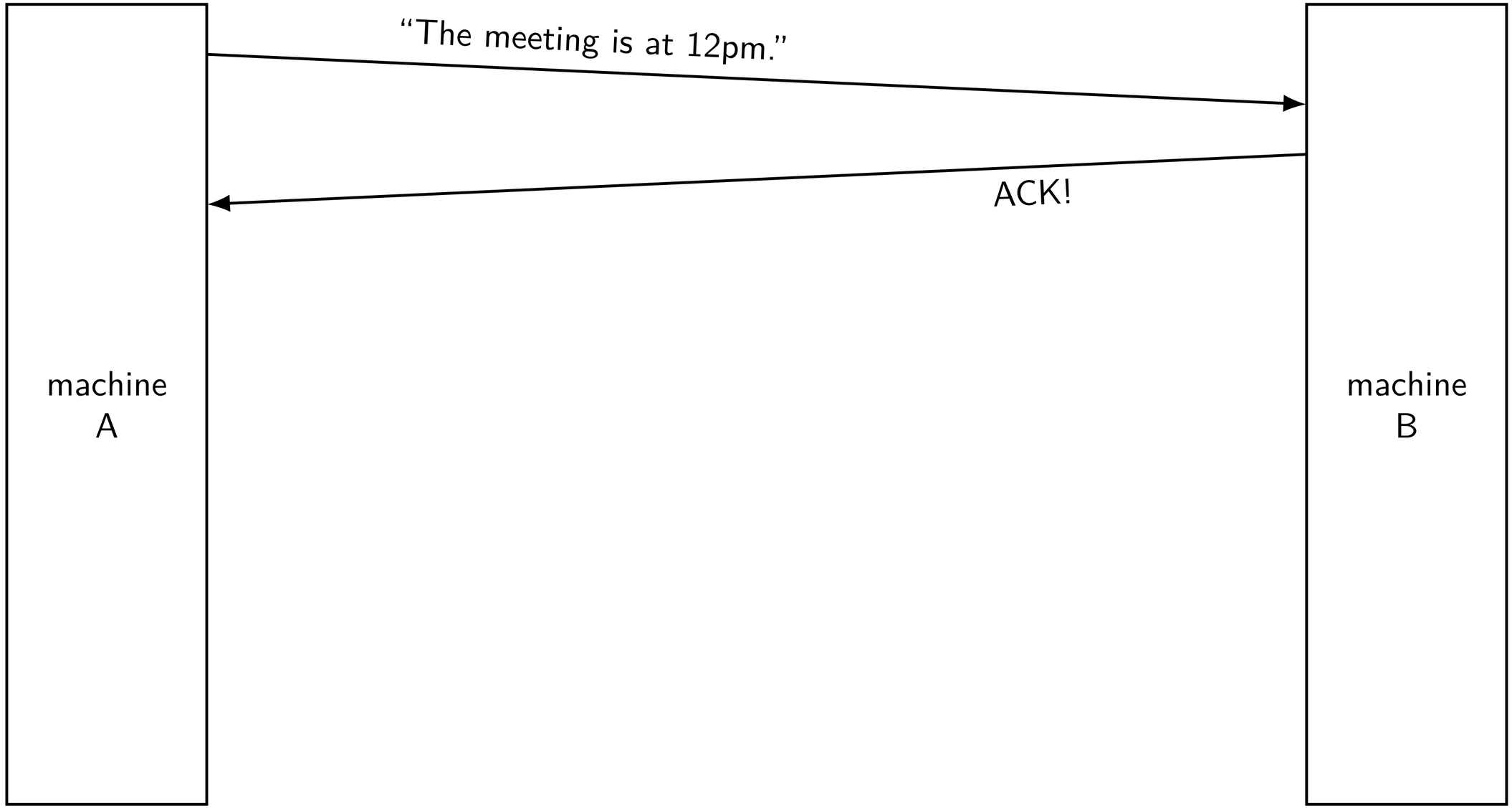
dealing with network message lost

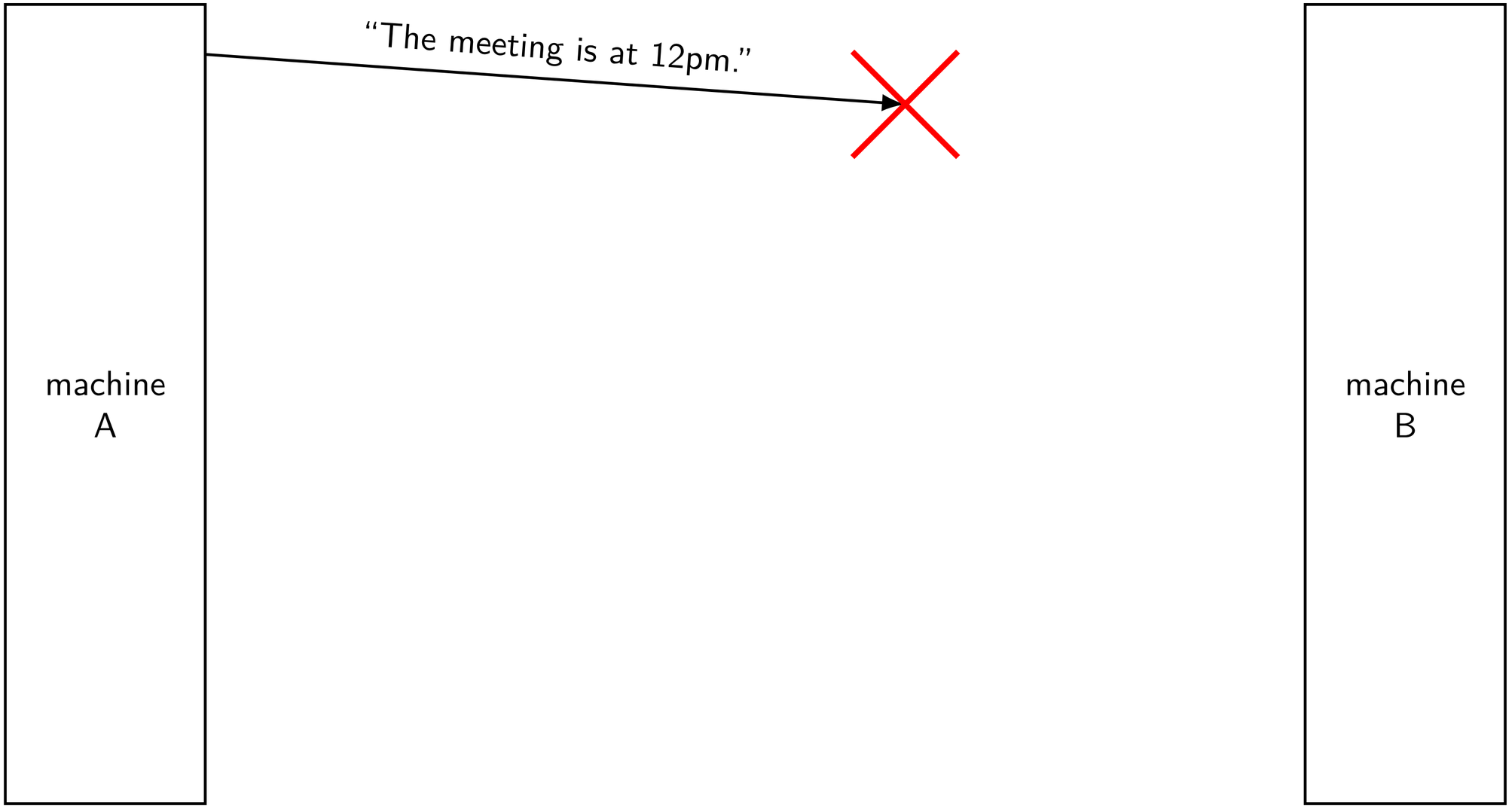


handling lost message: acknowledgements

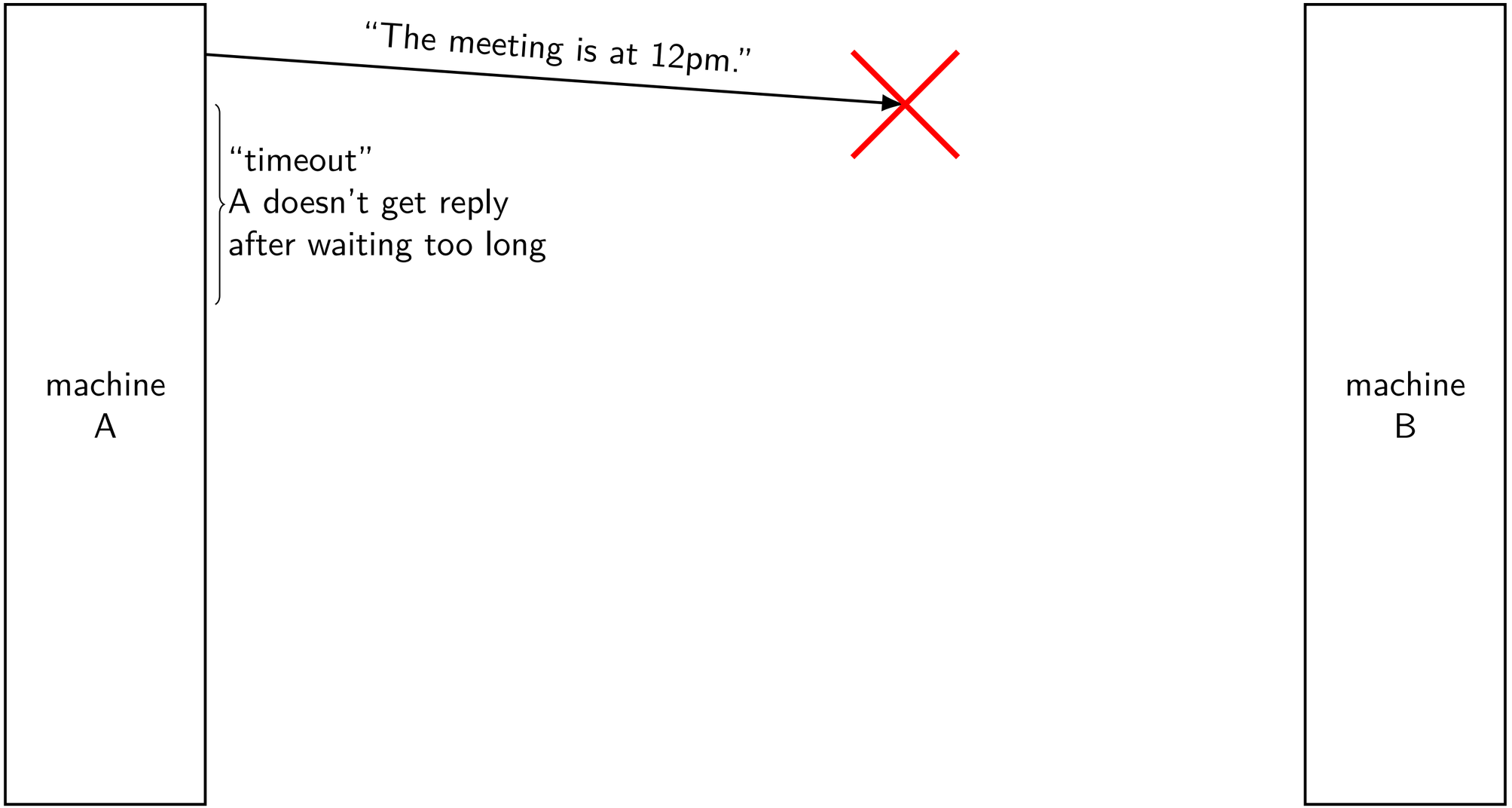


handling lost message

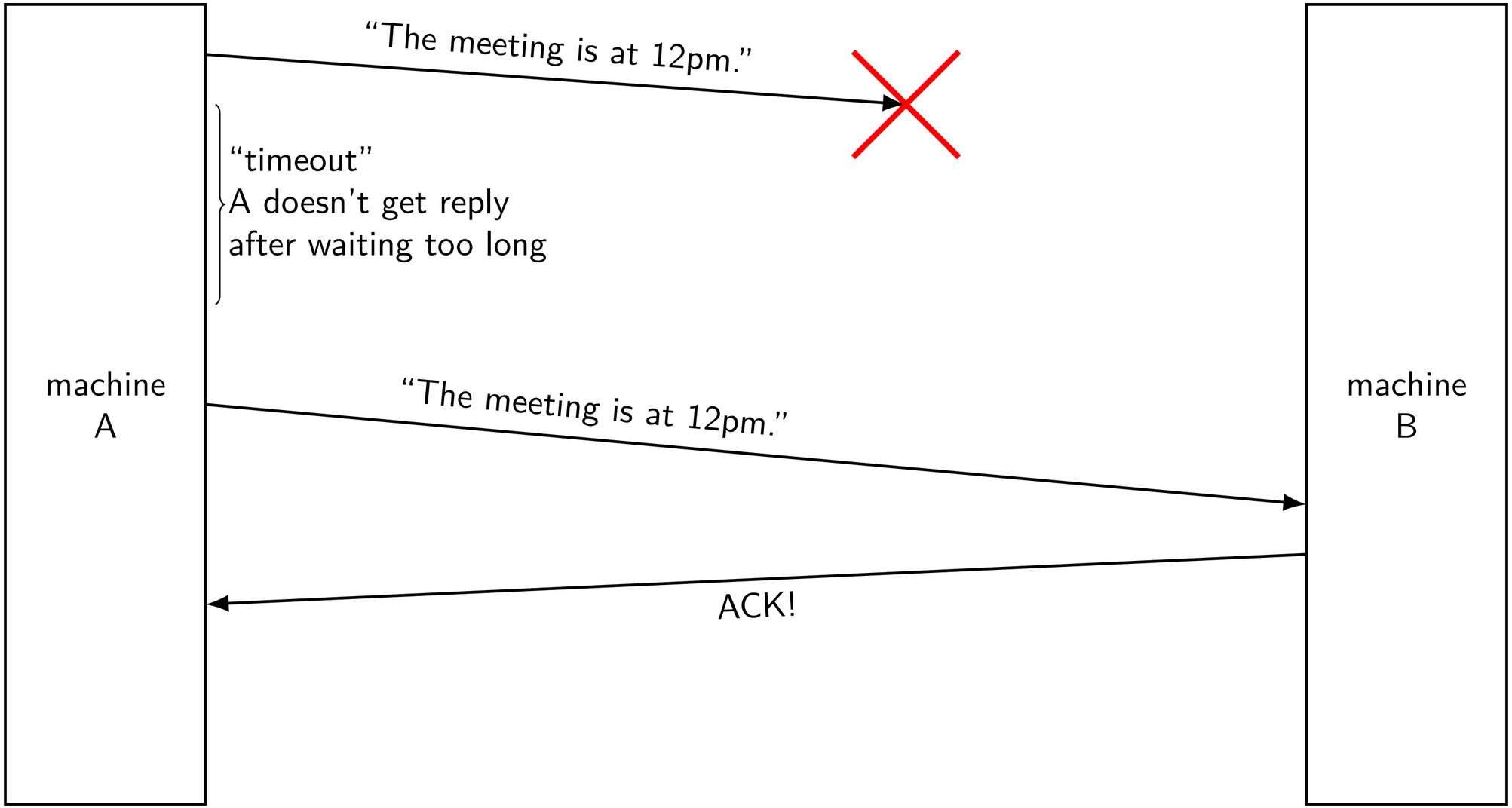
handling lost message



handling lost message



handling lost message



protocol so far

on sender: until ACK received:

(re)send frame of data

wait fixed amount of time for ACK

on receiver: continuously:

wait for frame of data

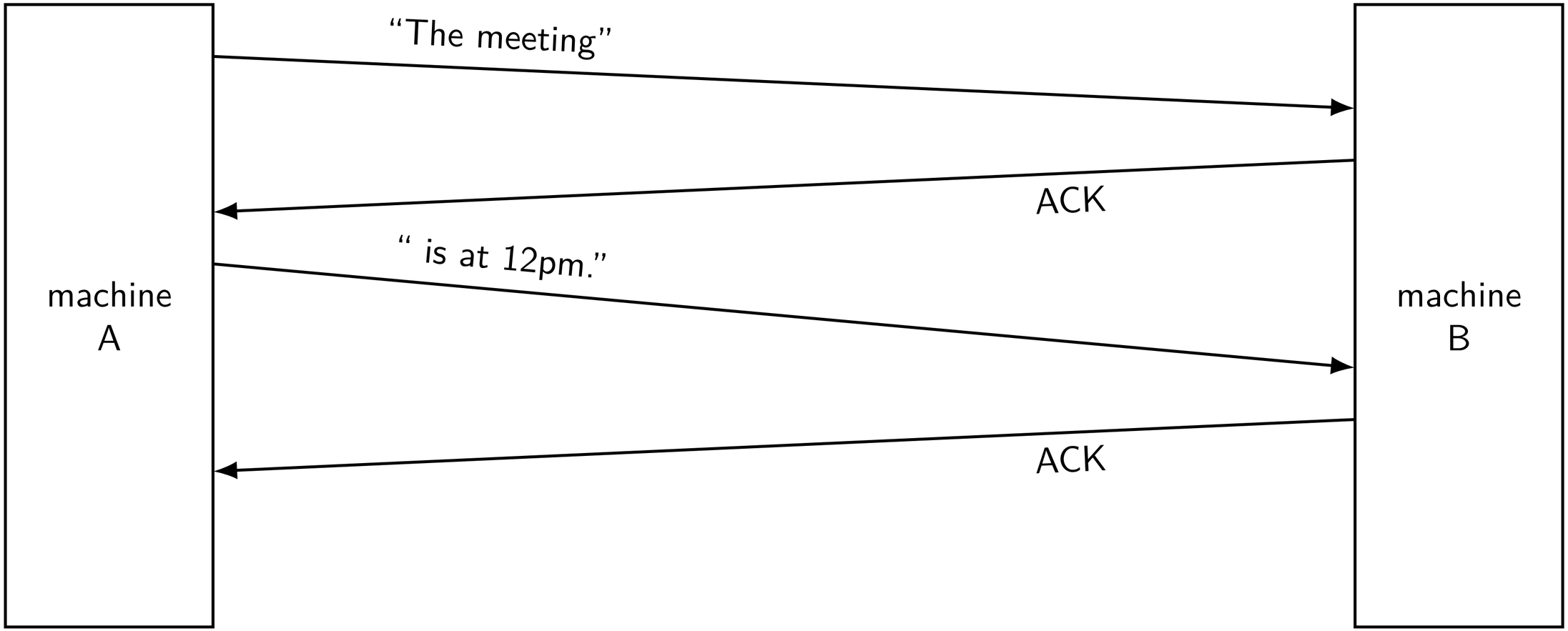
send ACK back

problem

really want to send multiple frames

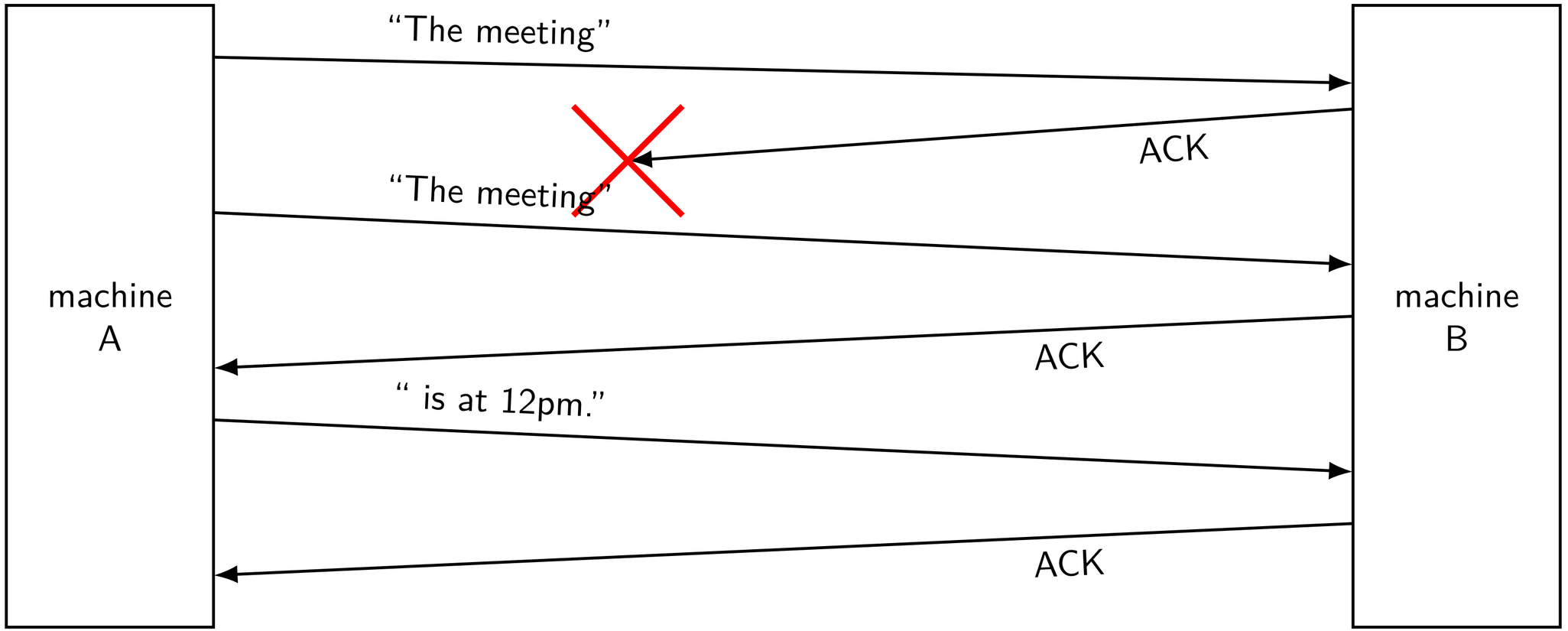
example: data split in multiple pieces

splitting messages: try 1

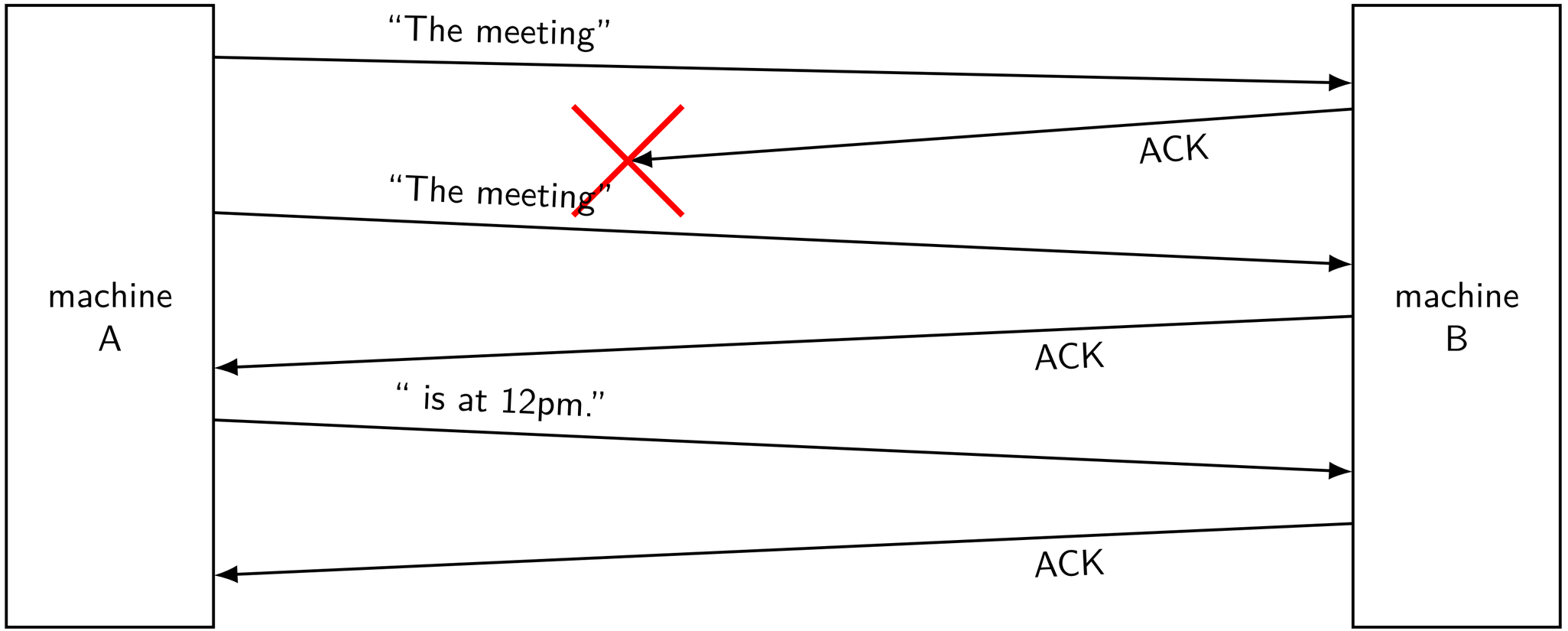


reconstructed message:
The meeting is at 12pm.

splitting messages: try 1 — problem 1



splitting messages: try 1 — problem 1



reconstructed message:
The meetingThe meeting is at 12pm.

exercise: other problems?

sending ‘The meeting’, ‘is at 12pm’

what would be received for each of these scenarios?

1. message (instead of acknowledgment) is lost
2. first message from machine A is delayed a long time by network
3. acknowledgment of second message lost instead of first

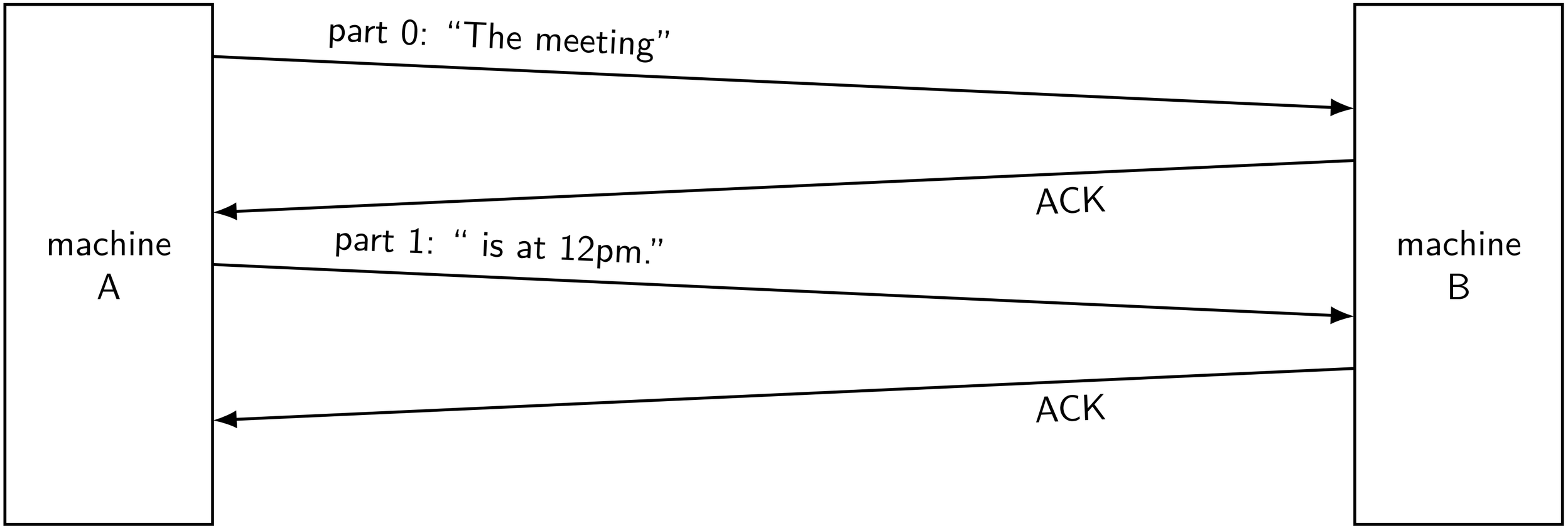
aside: message delays

long message delays not possible with direct link

but are possible with:

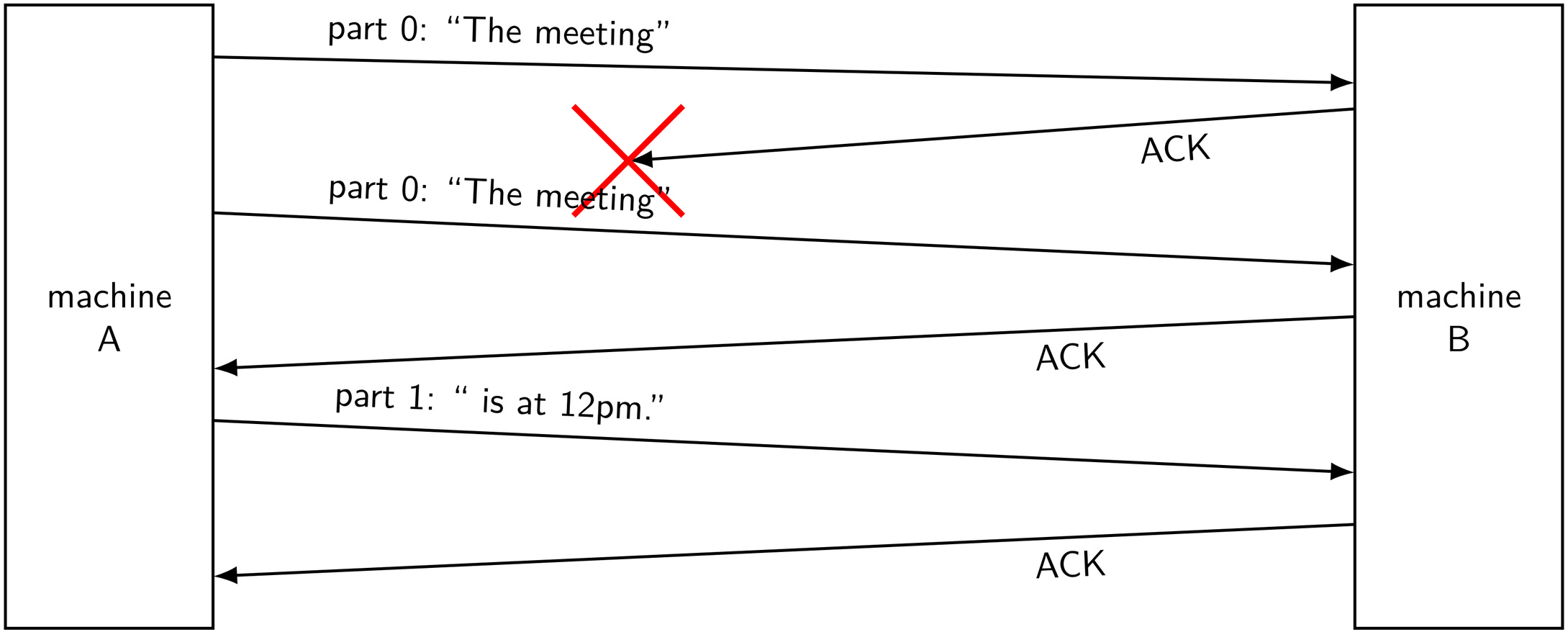
- multiple paths from A to B
- doing this kind of acknowledgment + resending hop-by-hop

splitting messages: try 2



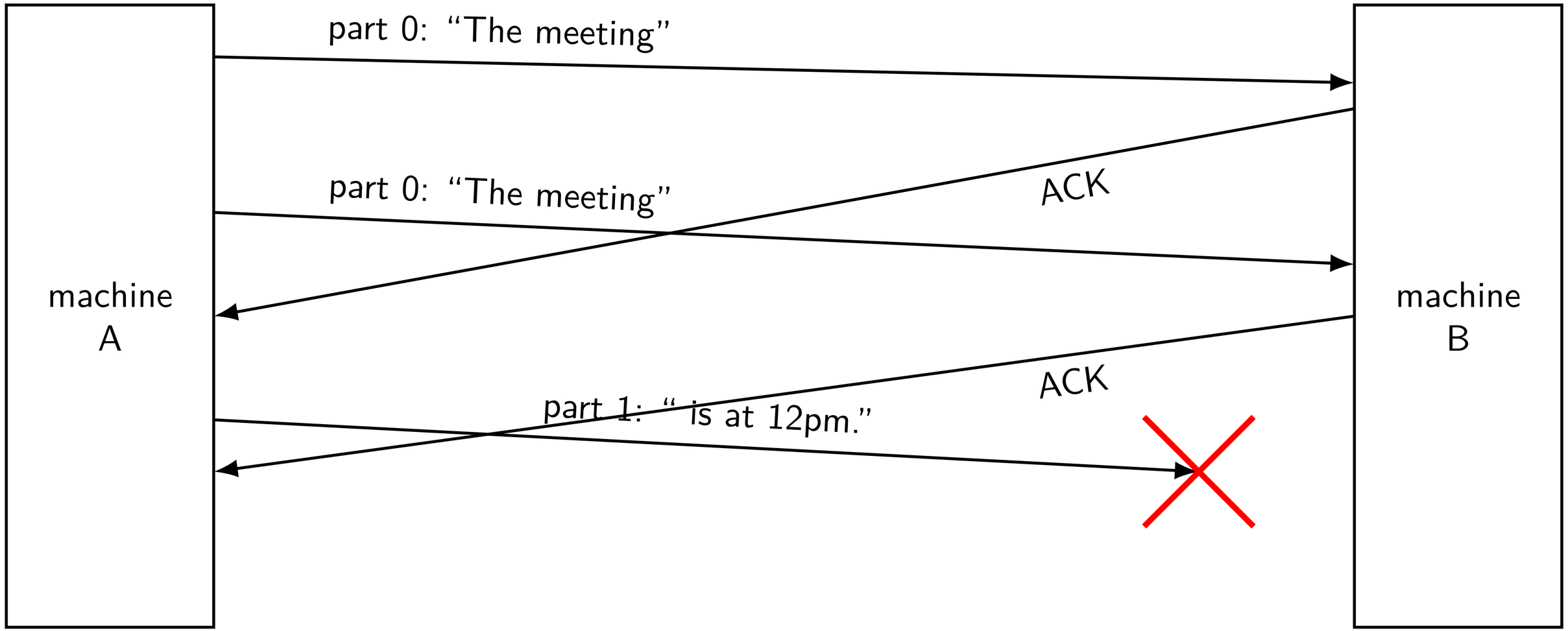
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — missed ack



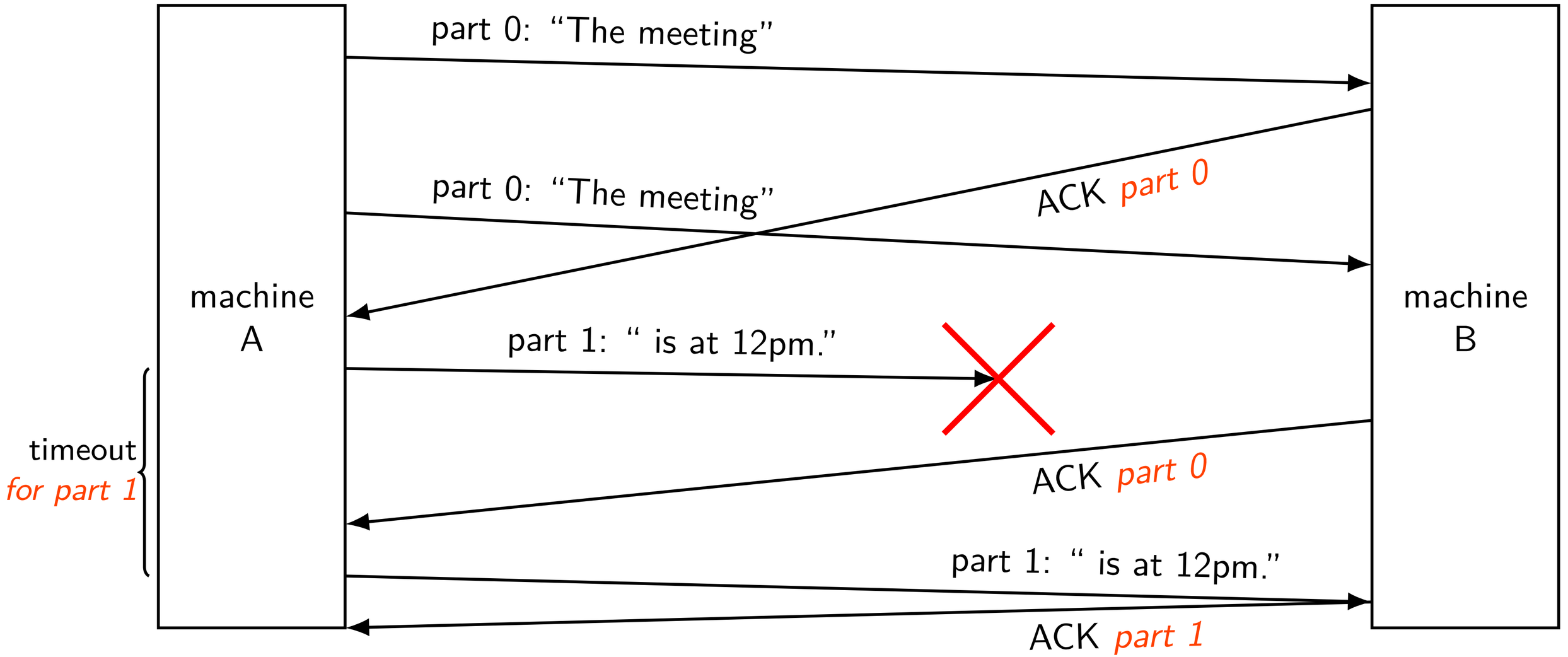
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — problem



A thinks: part 0 + part 1 acknowleged!

splitting messages: version 3



sequence numbers

call the ‘part’ label *sequence number*

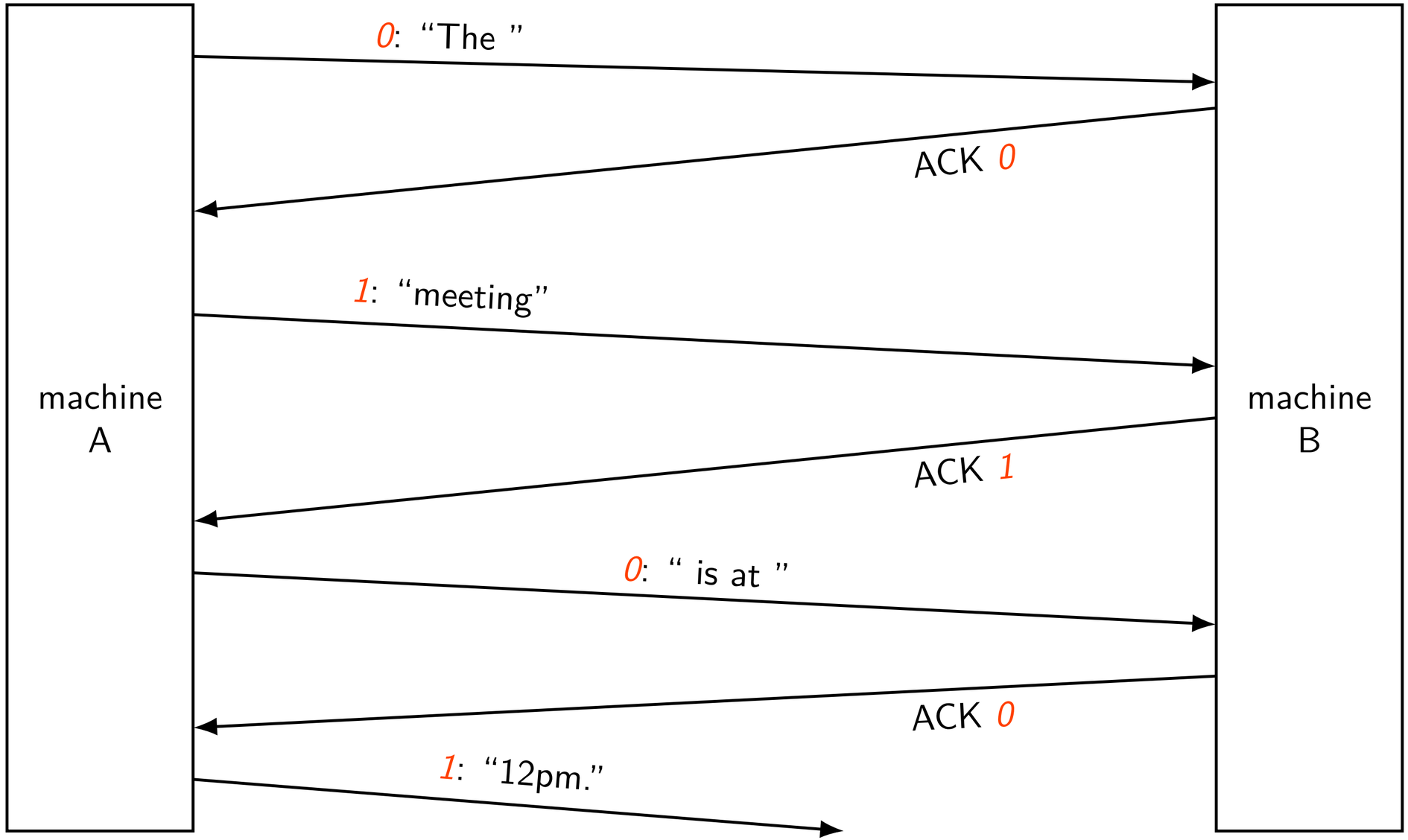
- for now: sequence number = message (or *segment*) number
- in TCP: sequence number = byte number

important question: how large can they get?

if we never reuse them — infinite!

so *really* want to reuse them

1-bit sequence number



'stop and wait'

machine A is only sending *one thing at a time*

never start sending next thing until after sending previous thing

stop-and-wait exercise (receive, 1)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `0: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (receive, 2)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `1: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (receive, 3)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `1: Y`

machine B sends `ACK 1`

machine B receives `0: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (send, 1)

A trying to send ‘X’, then ‘Y’, then ‘Z’

machine A sends `0: X`

machine A sends `0: X`

machine A receives `ACK 0`

machine A sends `1: Y`

machine A receives `ACK 0`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

stop-and-wait exercise (send, 2)

A trying to send ‘X’, then ‘Y’, then ‘Z’

machine A sends `0: X`

machine A sends `0: X`

machine A receives `ACK 0`

machine A sends `1: Y`

machine A receives `ACK 1`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

stop-and-wait issues

two issues with stop-and-wait:

doesn’t use close to full capacity of network

not clear how to set timeouts

looking at metrics

several important metrics we'll care about

(both for this and future topics)

throughput and *bandwidth* ($\sim$ how much capacity used/available)

latency and *round-trip time* (RTT) ($\sim$ what timeouts needed)

jitter ($\sim$ safety margin for timeouts)

stop-and-wait performance

stop-and-wait protocol

assuming no packets lost/corrupted

about *one packet per round-trip time*

(assuming packet tranmission time short relative to round-trip time)

example: local ethernet

my home wired network: 0.6 ms round trip time

typical packet has about 1400 bytes = 11200 bits of data

throughput with stop-and-wait:
$11200\mathrm{b}/0.6\mathrm{ms} \approx 19000\mathrm{b}/\mathrm{ms} = 19\ 000\ 000\mathrm{b}/\mathrm{s} = 19\mathrm{Mbit}/\mathrm{s}$

available bandwidth is about 1 Gbit/s

example: local ethernet

my home wired network: 0.6 ms round trip time

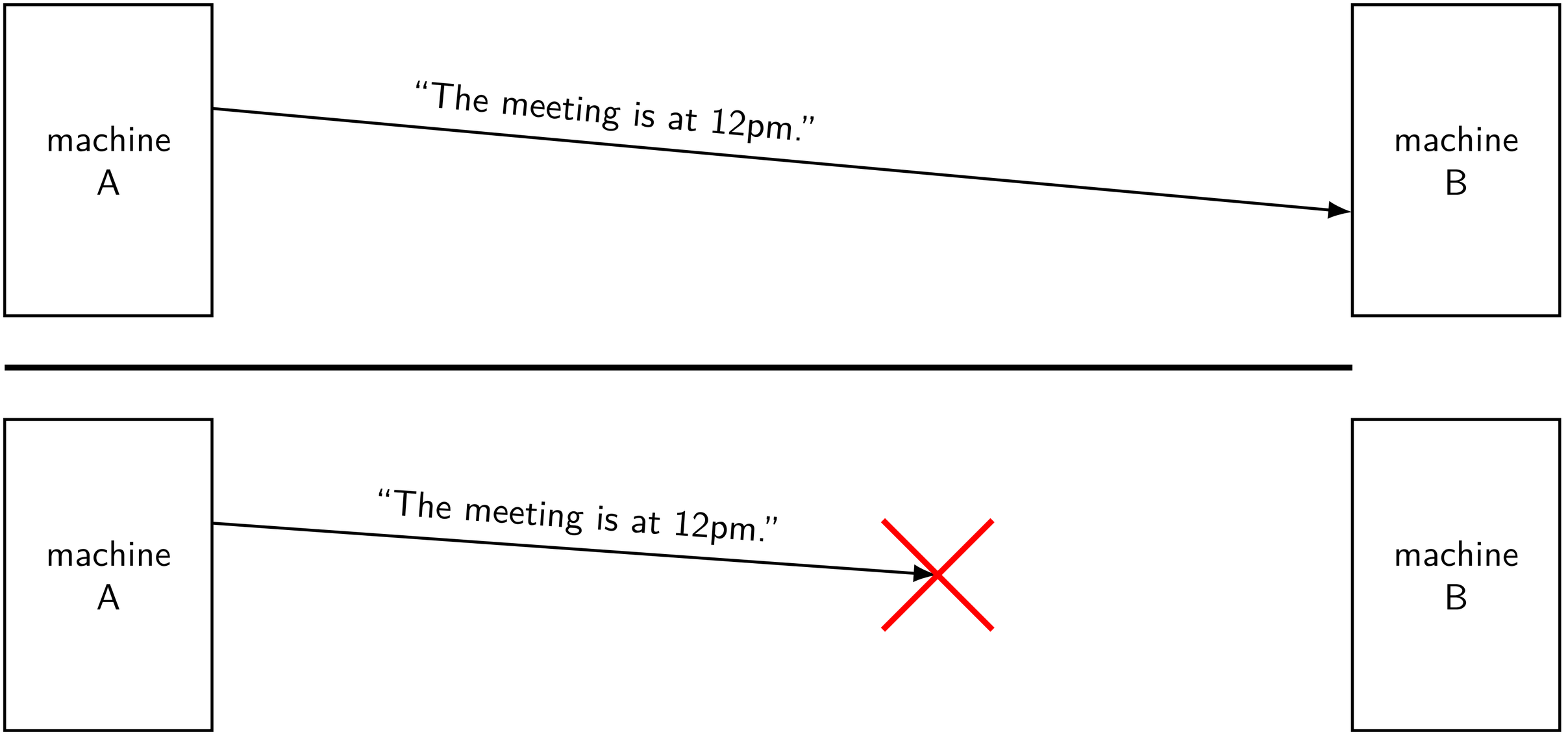
typical packet has about 1400 bytes = 11200 bits of data

throughput with stop-and-wait:
$11200\mathrm{b}/0.6\mathrm{ms} \approx 19000\mathrm{b}/\mathrm{ms} = 19\ 000\ 000\mathrm{b}/\mathrm{s} = 19\mathrm{Mbit}/\mathrm{s}$

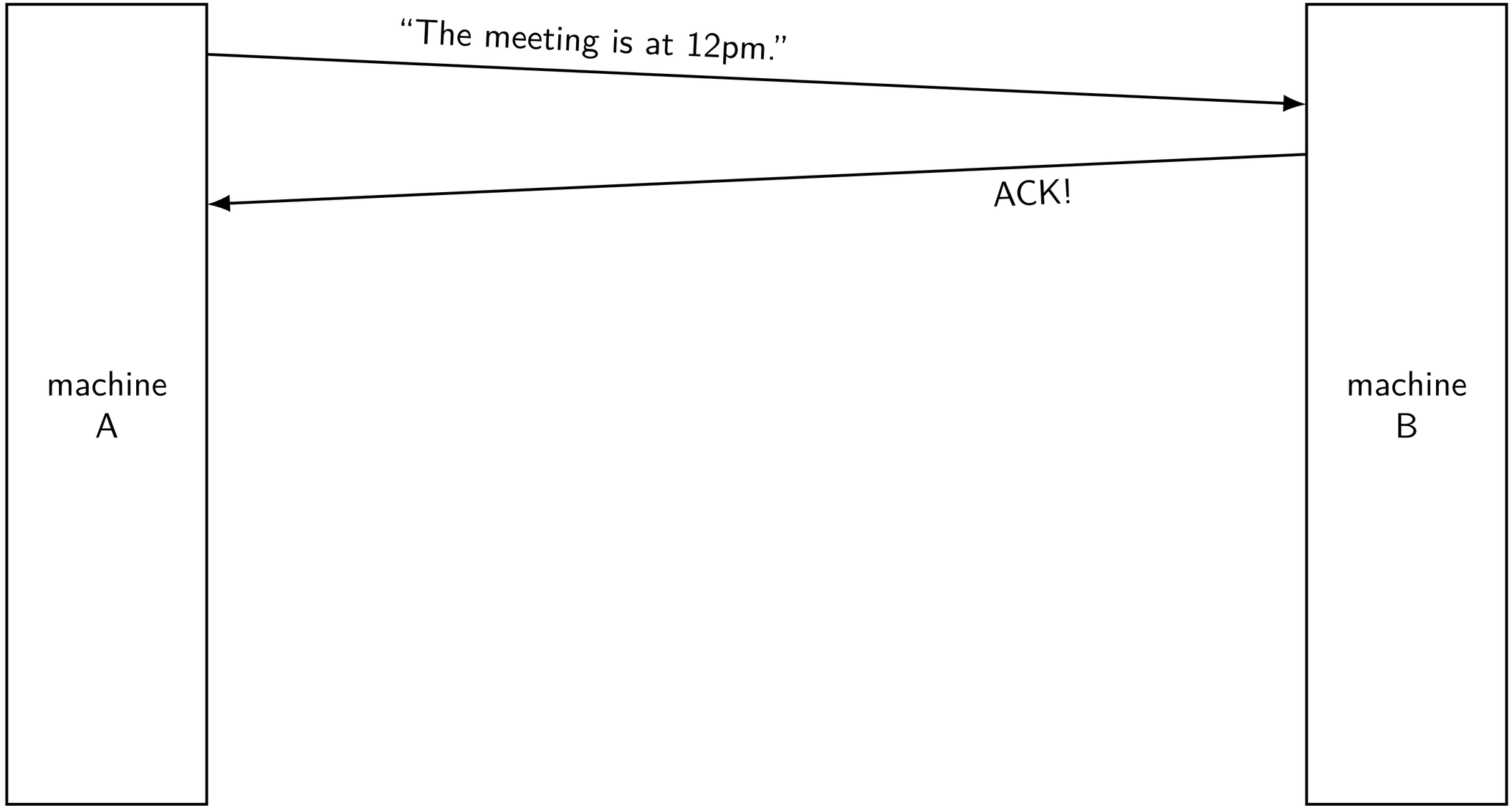
available bandwidth is about 1 *Gbit/s*

backup slides

dealing with network message lost

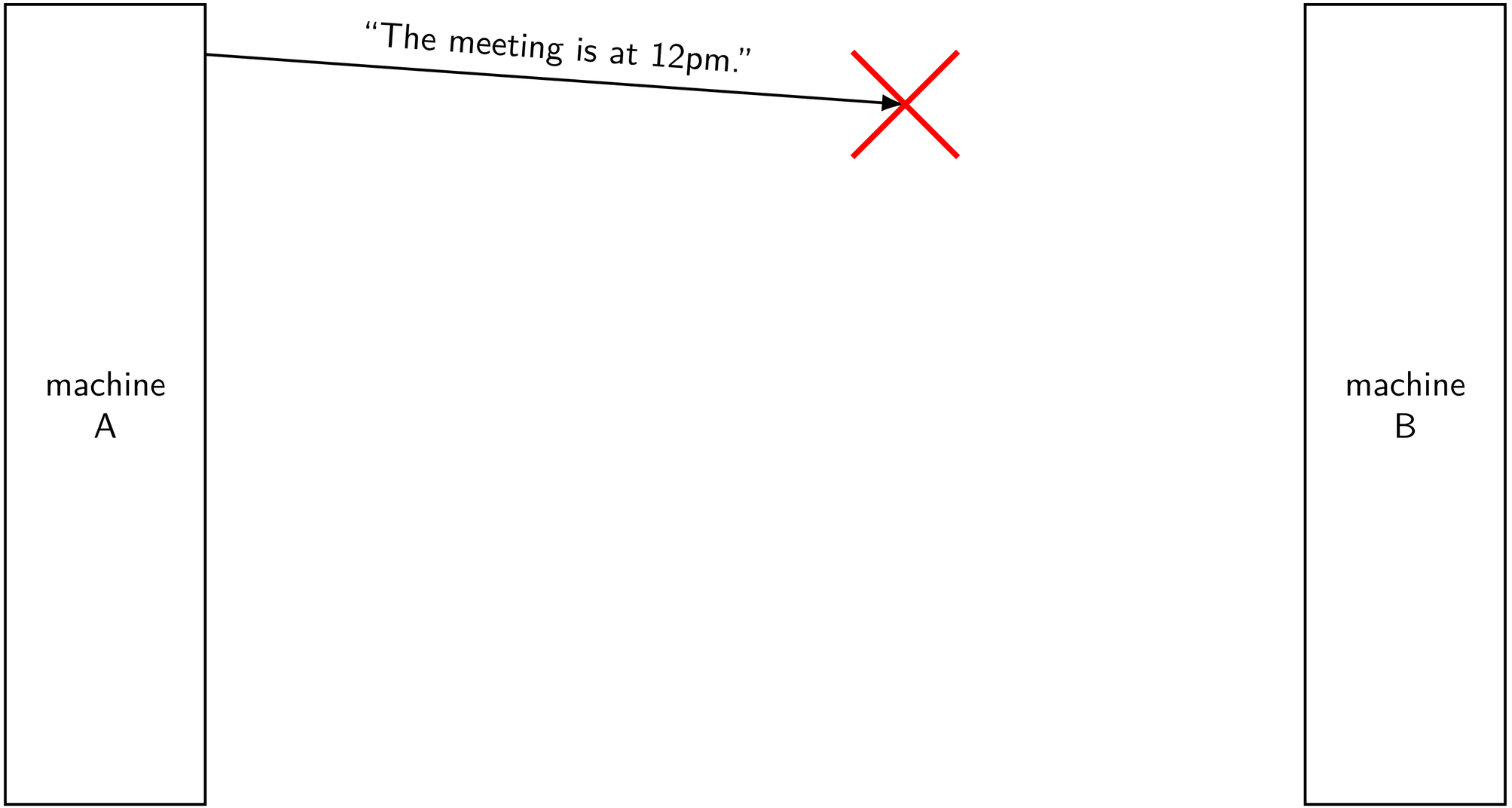


handling lost message: acknowledgements

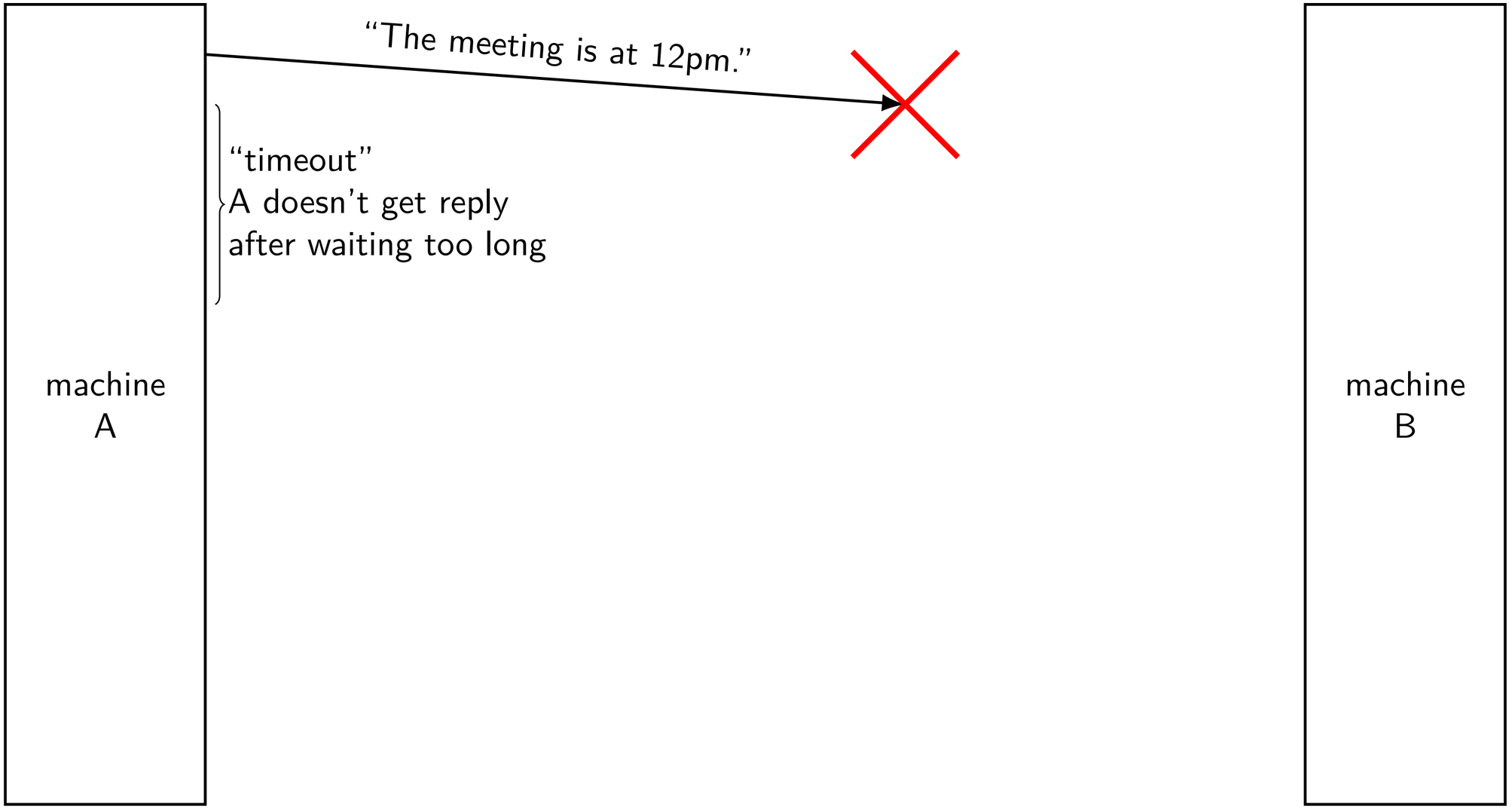


handling lost message

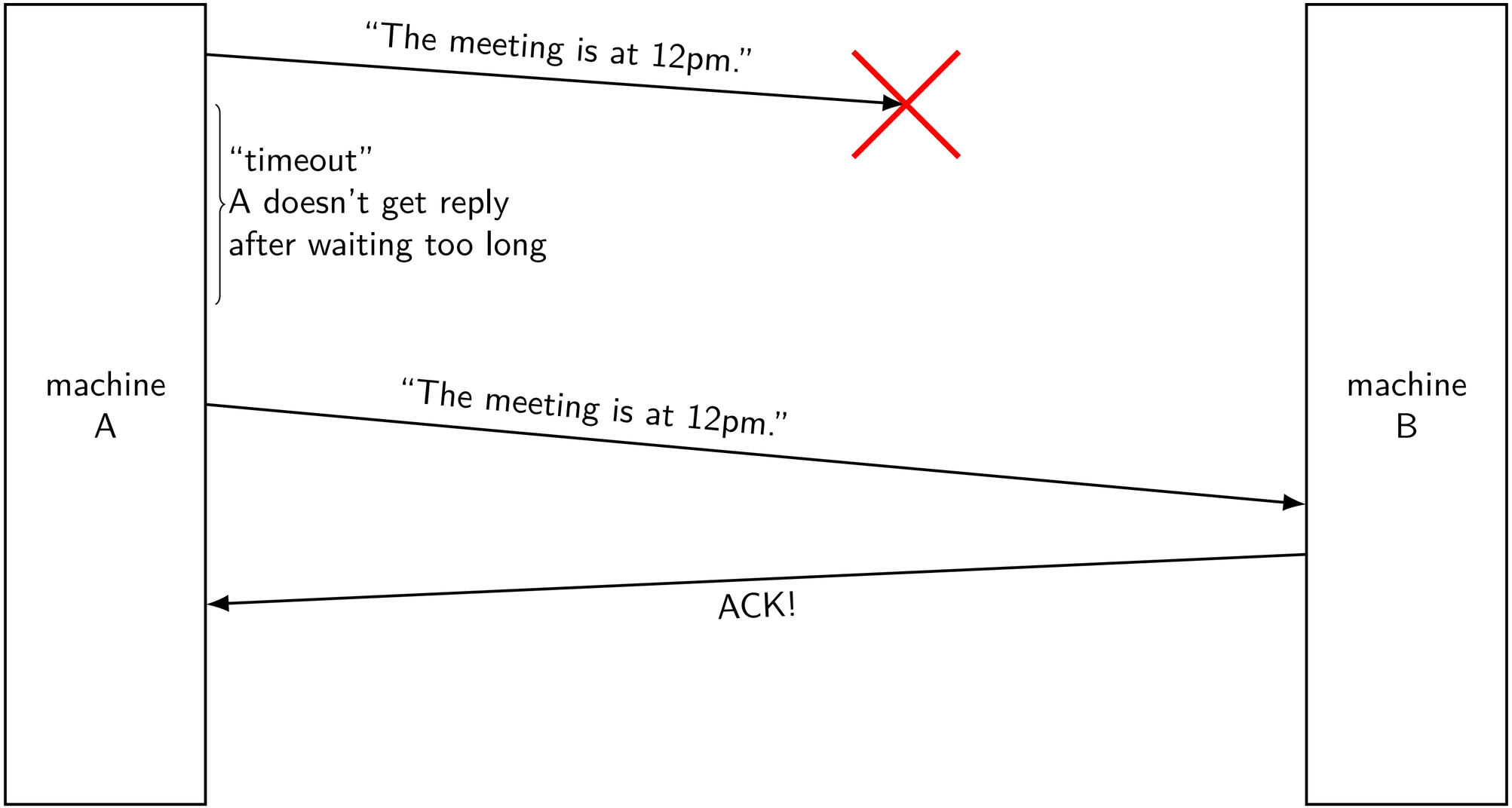
handling lost message



handling lost message



handling lost message



protocol so far

on sender: until ACK received:

(re)send frame of data

wait fixed amount of time for ACK

on receiver: continuously:

wait for frame of data

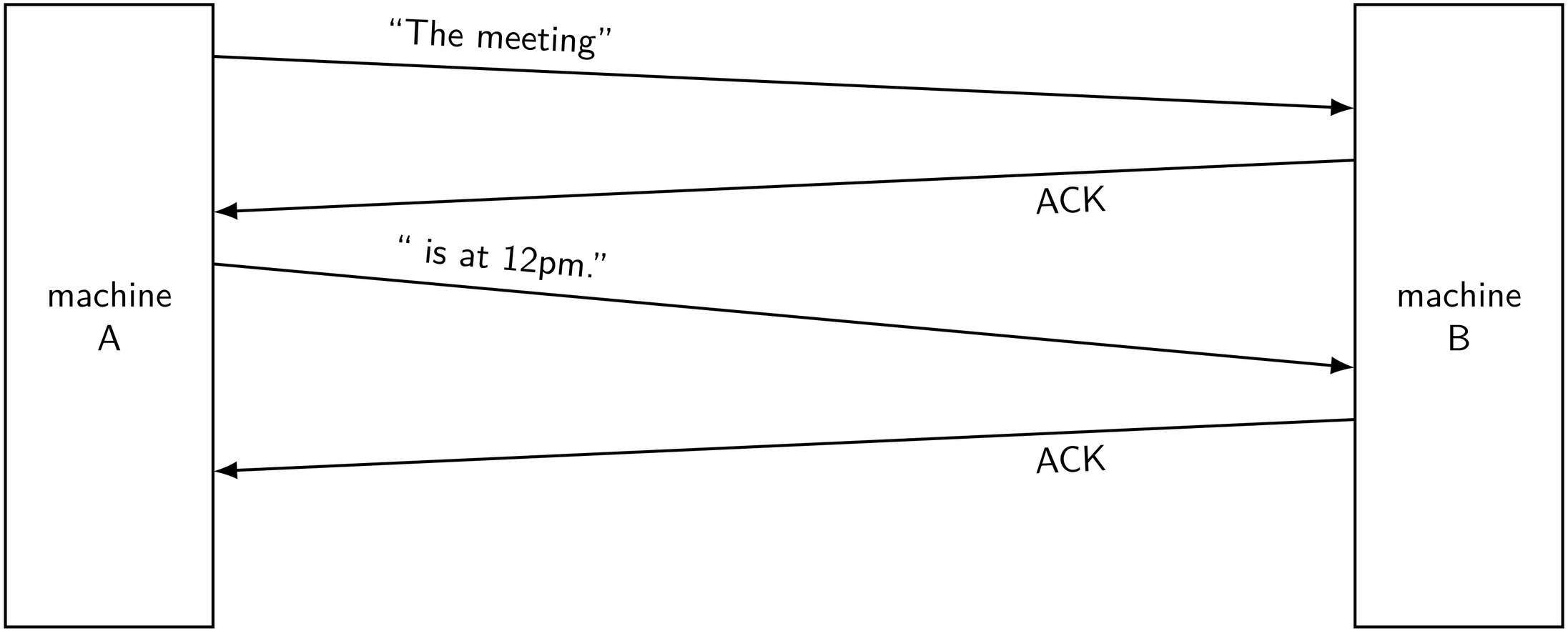
send ACK back

problem

really want to send multiple frames

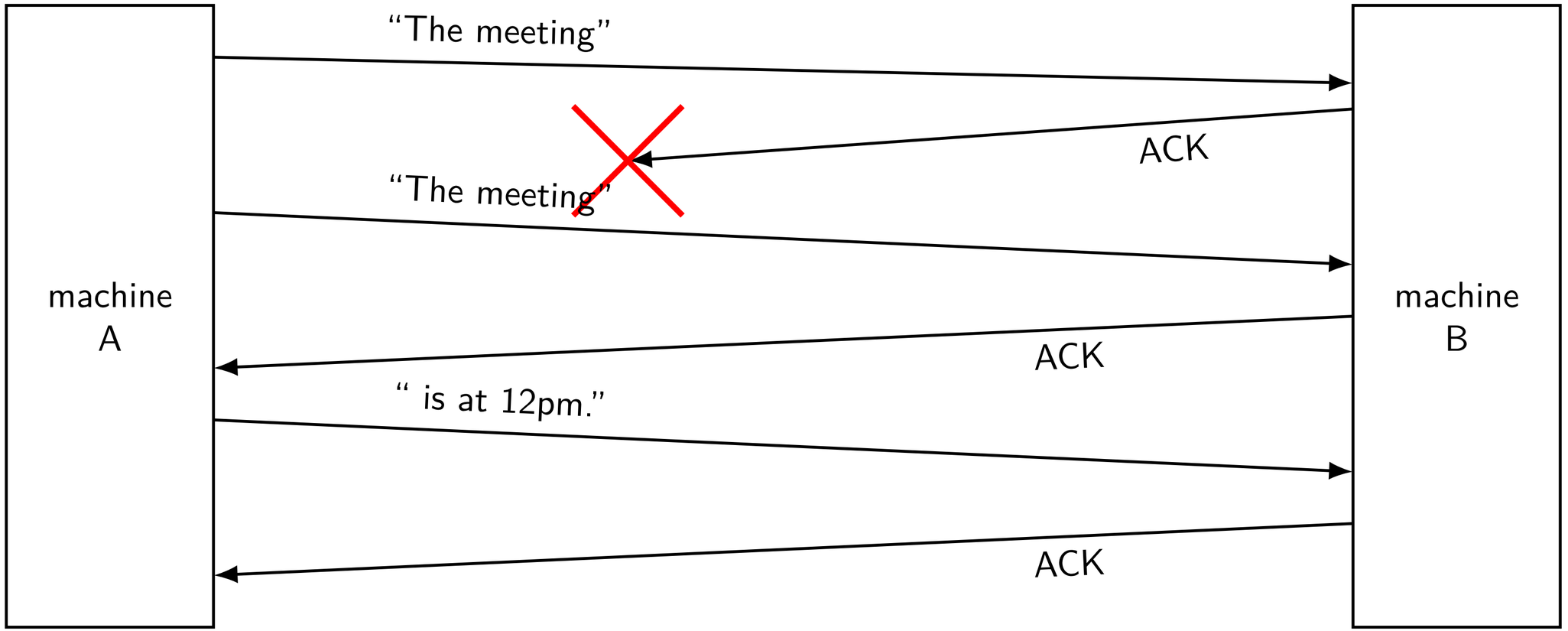
example: data split in multiple pieces

splitting messages: try 1

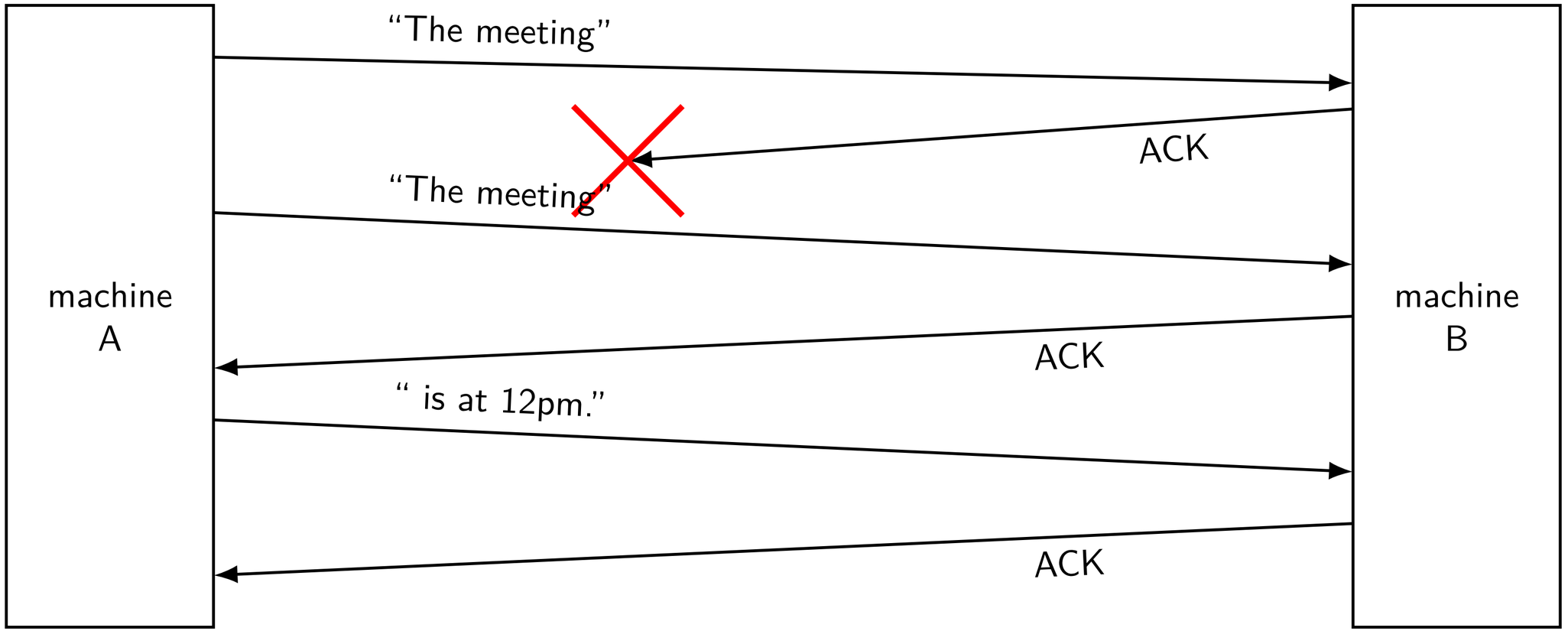


reconstructed message:
The meeting is at 12pm.

splitting messages: try 1 — problem 1



splitting messages: try 1 — problem 1



reconstructed message:
The meetingThe meeting is at 12pm.

exercise: other problems?

sending ‘The meeting’, ‘is at 12pm’

what would be received for each of these scenarios?

1. message (instead of acknowledgment) is lost
2. first message from machine A is delayed a long time by network
3. acknowledgment of second message lost instead of first

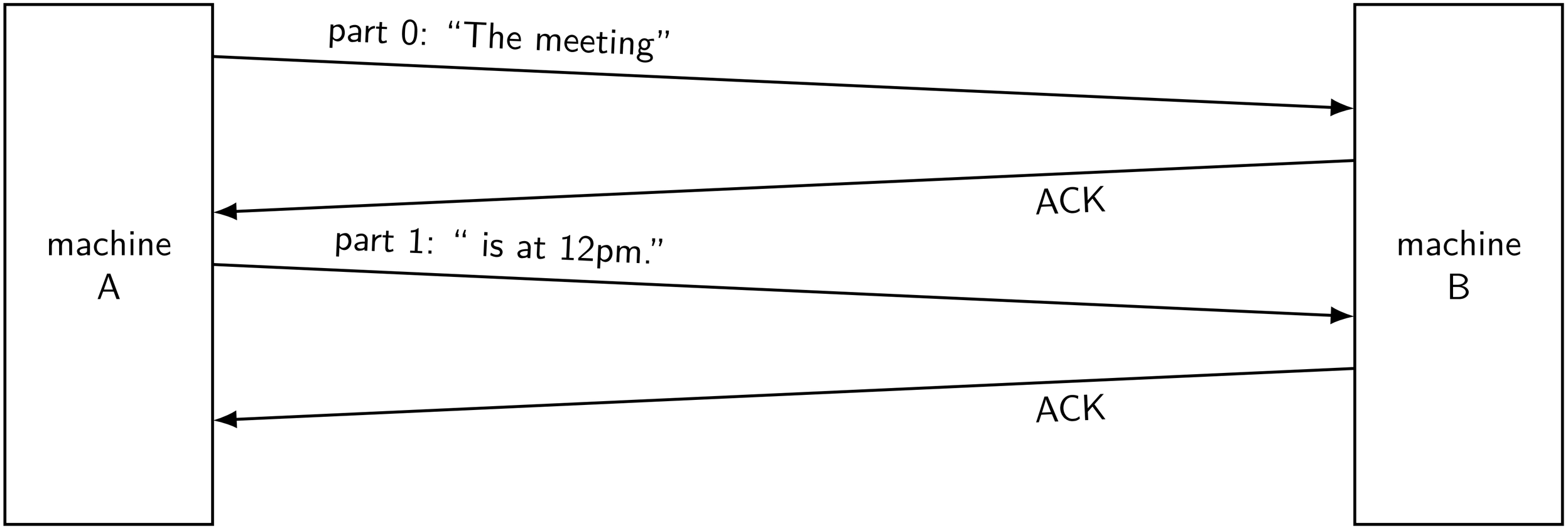
aside: message delays

long message delays not possible with direct link

but are possible with:

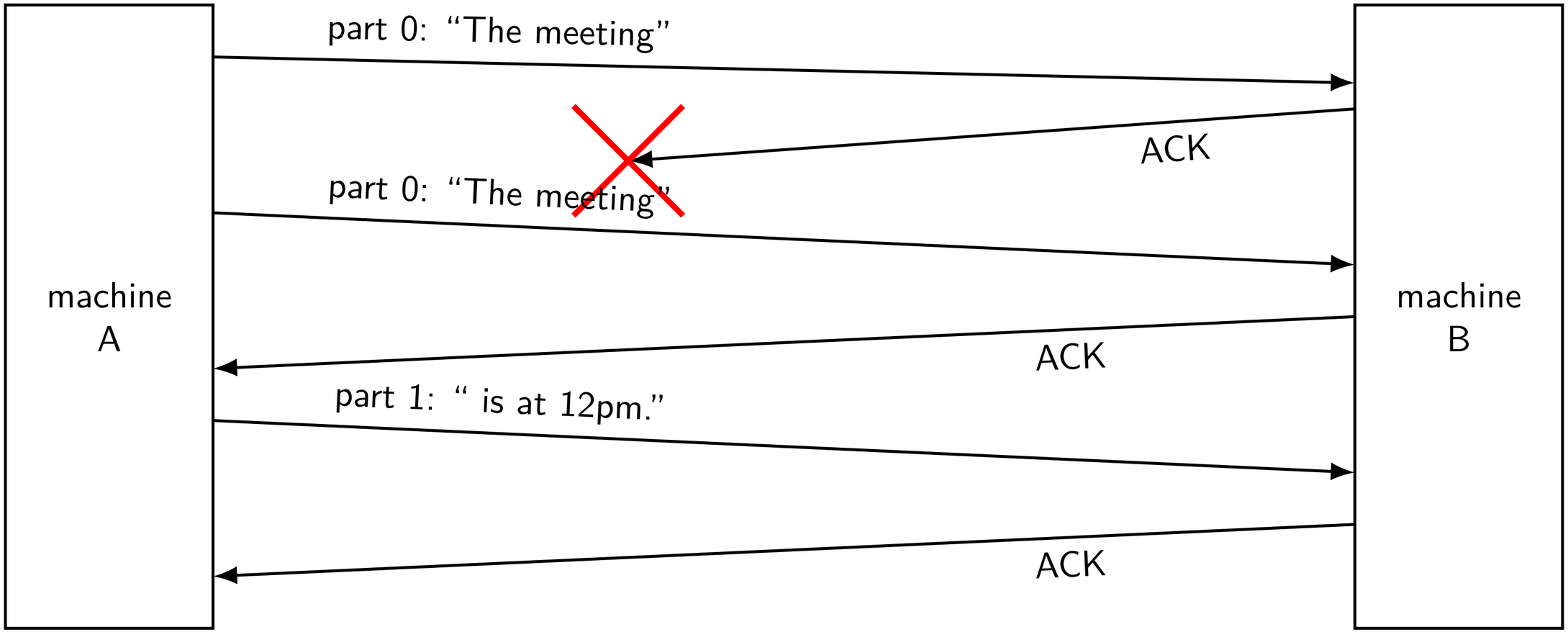
- multiple paths from A to B
- doing this kind of acknowledgment + resending hop-by-hop

splitting messages: try 2



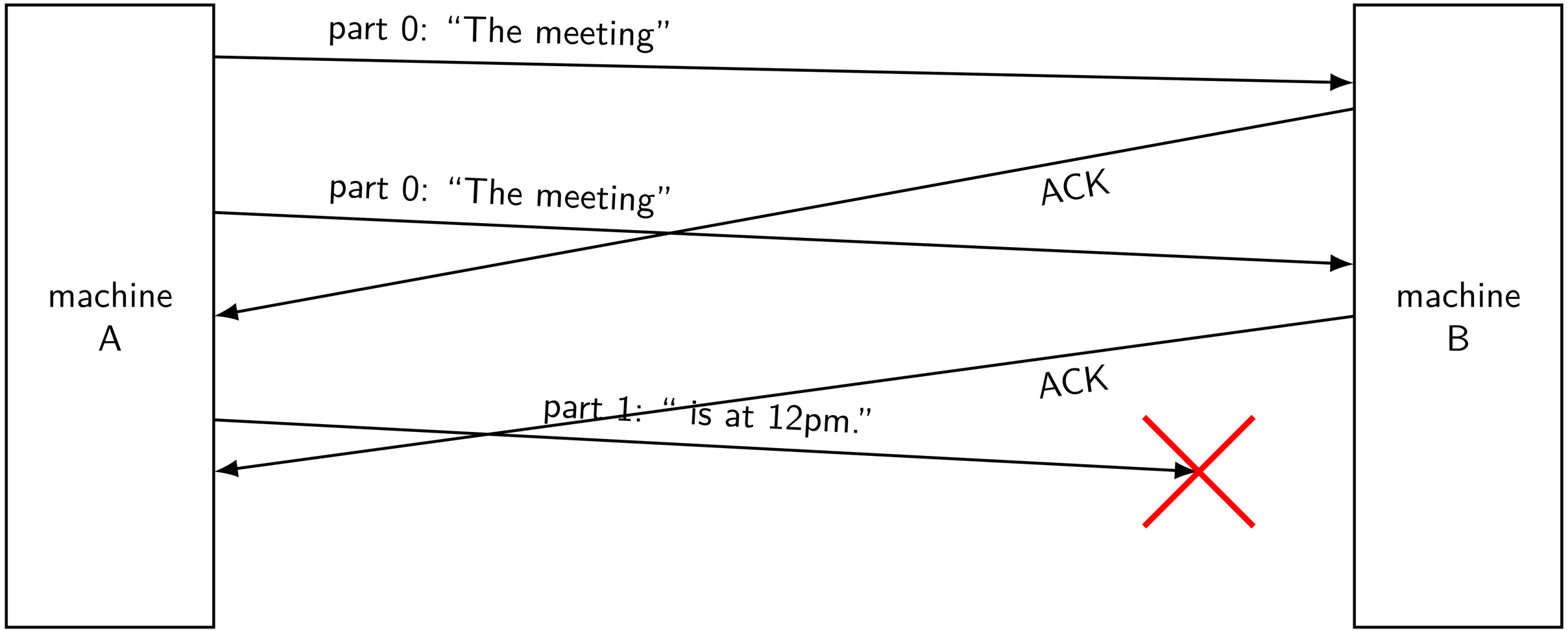
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — missed ack



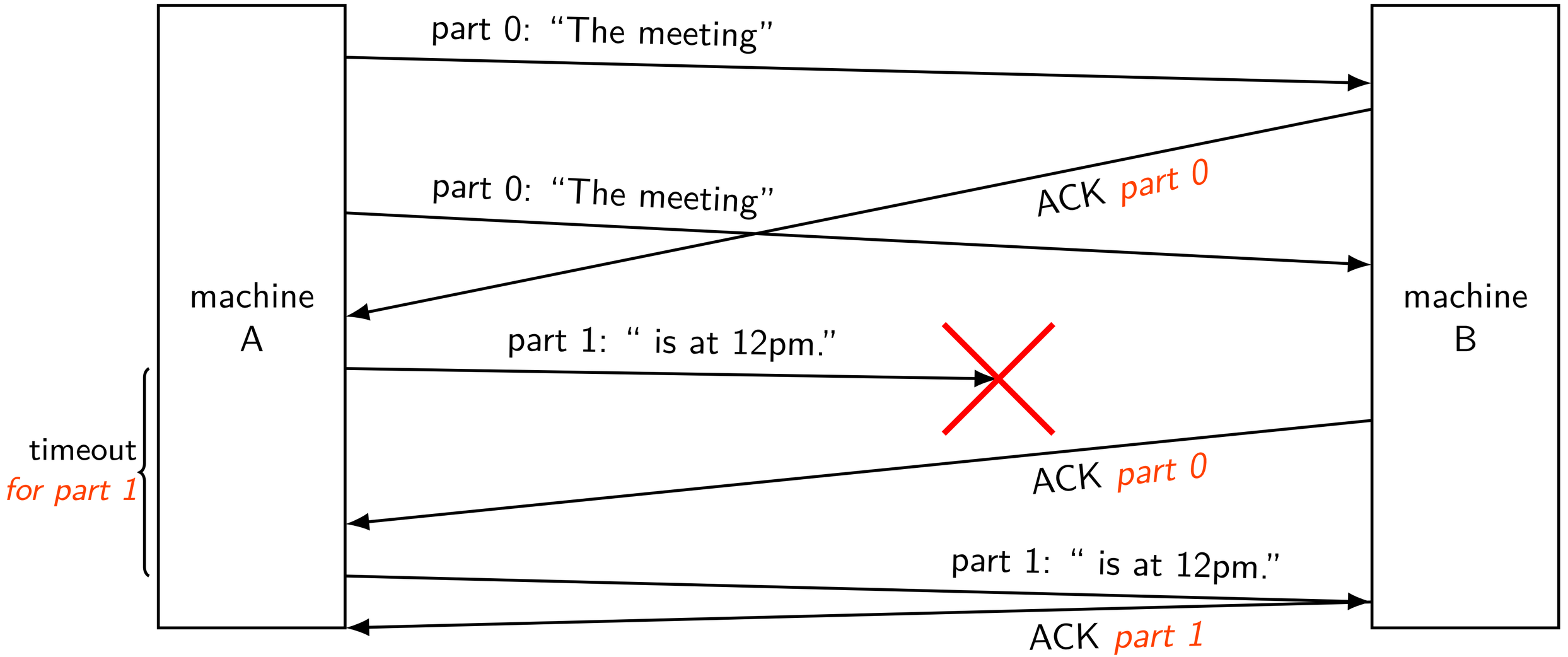
reconstructed message:
The meeting is at 12pm.

splitting messages: try 2 — problem



A thinks: part 0 + part 1 acknowleged!

splitting messages: version 3



sequence numbers

call the ‘part’ label *sequence number*

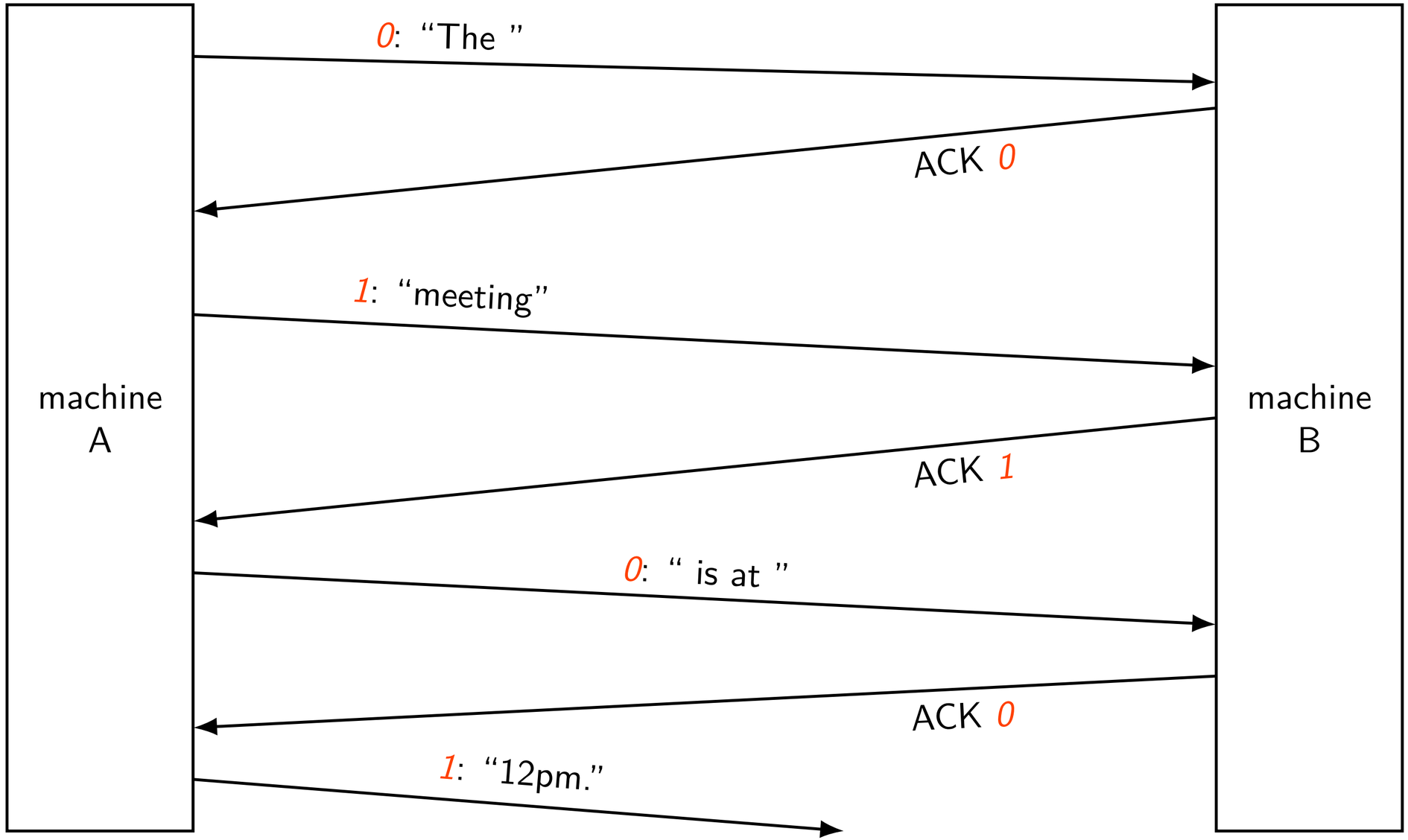
- for now: sequence number = message (or *segment*) number
- in TCP: sequence number = byte number

important question: how large can they get?

if we never reuse them — infinite!

so *really* want to reuse them

1-bit sequence number



'stop and wait'

machine A is only sending *one thing at a time*

never start sending next thing until after sending previous thing

stop-and-wait exercise (receive, 1)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `0: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (receive, 2)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `1: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (receive, 3)

machine B receives `0: X`

machine B sends `ACK 0`

machine B receives `1: Y`

machine B sends `ACK 1`

machine B receives `0: X`

what should machine B do now?

A. send `ACK 0` B. send `ACK 1` C. send nothing

stop-and-wait exercise (send, 1)

A trying to send ‘X’, then ‘Y’, then ‘Z’

machine A sends `0: X`

machine A sends `0: X`

machine A receives `ACK 0`

machine A sends `1: Y`

machine A receives `ACK 0`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

stop-and-wait exercise (send, 2)

A trying to send ‘X’, then ‘Y’, then ‘Z’

machine A sends `0: X`

machine A sends `0: X`

machine A receives `ACK 0`

machine A sends `1: Y`

machine A receives `ACK 1`

what should machine A do now?

A. send `0: X` again B. send `1: Y` again

C. send `0: Z` D. something else

stop-and-wait issues

two issues with stop-and-wait:

doesn’t use close to full capacity of network

not clear how to set timeouts

looking at metrics

several important metrics we'll care about

(both for this and future topics)

throughput and *bandwidth* ($\sim$ how much capacity used/available)

latency and *round-trip time* (RTT) ($\sim$ what timeouts needed)

jitter ($\sim$ safety margin for timeouts)

stop-and-wait performance

stop-and-wait protocol

assuming no packets lost/corrupted

about *one packet per round-trip time*

(assuming packet tranmission time short relative to round-trip time)

example: local ethernet

my home wired network: 0.6 ms round trip time

typical packet has about 1400 bytes = 11200 bits of data

throughput with stop-and-wait:
$11200\mathrm{b}/0.6\mathrm{ms} \approx 19000\mathrm{b}/\mathrm{ms} = 19\ 000\ 000\mathrm{b}/\mathrm{s} = 19\mathrm{Mbit}/\mathrm{s}$

available bandwidth is about 1 Gbit/s

example: local ethernet

my home wired network: 0.6 ms round trip time

typical packet has about 1400 bytes = 11200 bits of data

throughput with stop-and-wait:
$11200\mathrm{b}/0.6\mathrm{ms} \approx 19000\mathrm{b}/\mathrm{ms} = 19\ 000\ 000\mathrm{b}/\mathrm{s} = 19\mathrm{Mbit}/\mathrm{s}$

available bandwidth is about 1 *Gbit/s*